

部署指南 (render 主交付 / AWS 計畫書完整版)

兩套部署並存 (D005) : **render** 對齊作業評分要求、**AWS** 對齊計畫書原案。

A. render (推薦、主交付)

整個 `render.yaml` 已定義好 : PostgreSQL + backend-api + backend-node + frontend 。

1. 把 repo 連到 render (New → Blueprint , 選此 repo) , render 讀 `render.yaml` 自動建 4 個服務。
2. 在 `backend-api` 服務的 Environment 填 `GEMINI_API_KEY` (啟用 Gemini ; 不填走規則+模型) 。
3. 首次部署後 , 把資料灌進 DB :

```
bash psql "<render External Database URL>" -f db/schema.sql psql "<render External Database URL>" -f db/seed.sql # 真實資料：在本機設 DATABASE_URL=<render URL> 後執行  
DATABASE_URL="<render URL>" python -m crawler.run --gov
```
4. `backend-api` build 階段會自動 `python -m model.train` 訓練模型。
5. 前端環境變數 `VITE_API_URL` / `VITE_NODE_URL` 指向兩個後端服務網址 (在 render 前端服務設定 , 或 build 時帶入) 。

⚠ free plan : 服務閒置會休眠 (首次請求較慢) 、PostgreSQL 有期限 , demo/繳交前先喚醒並確認資料還在。

B. AWS (計畫書原案 : Amplify + Elastic Beanstalk + RDS)

B1. 資料庫 — RDS (PostgreSQL)

1. RDS → Create database → PostgreSQL , 記下 endpoint/帳密 , 組成 `DATABASE_URL` 。
2. 灌 schema/seed/真實資料 (同上 , psql 指向 RDS endpoint) 。

B2. 兩個後端 — Elastic Beanstalk (Docker)

兩後端各有 `Dockerfile` 。注意：build context 必須在 repo 根 (monorepo 相對路徑需 `db/`)：

```
# 本機先驗證可 build (從 repo 根)
docker build -f backend-api/Dockerfile -t scam-api .
docker build -f backend-node/Dockerfile -t scam-node .

# EB：在 repo 根放 Dockerrun/設定後部署，環境變數帶入連線字串
eb init -p docker scam-backend
eb create scam-api-env --envvars DATABASE_URL="...",GEMINI_API_KEY="..."
eb deploy
# backend-node 另建一環境 (不需 GEMINI_API_KEY)
```

B3. 前端 — Amplify Hosting

- 1. Amplify → Host web app → 連 repo · monorepo 設定 app root = `frontend` 。
- 2. Build： `npm install && npm run build`，輸出目錄 `dist` 。
- 3. 環境變數 `VITE_API_URL` / `VITE_NODE_URL` 指向兩個 EB 後端網址。

AWS 需注意免費額度與成本；若僅為「計畫書完整實現」展示，可短期開啟、demo 後關閉。

對照表

層	render	AWS
前端	Static Site	Amplify Hosting
後端 ×2	Web Service	Elastic Beanstalk (Docker)
資料庫	render PostgreSQL	RDS (PostgreSQL)
模型訓練	build 階段 <code>model.train</code>	Dockerfile build 階段